

Hevo Assessment 2

2. README.md

Hevo Assessment II: Post-Load with Hevo Models

2.1. Project Overview

- **Goal:** Set up a CDC-based data pipeline from a local PostgreSQL source to a Snowflake destination using Hevo Data, including data transformation.

2.2. Prerequisites & Accounts Used

- **Hevo Account Details:**
 - **Team Name:** siddharthkakade1999@gmail.com
 - **Pipeline ID:** #7
- **Local Postgres Setup:** Docker desktop running latest postgres image

2.3. Detailed Steps to Reproduce the Setup

2.3.1. Database Setup (PostgreSQL)

1. Installation:

Command ran in docker desktop:

```
docker run --name mydatabase2 -e POSTGRES_PASSWORD=admin -e POSTGRES_USER=admin -e POSTGRES_DB=mydatabase2 -p 5433:5432 -d postgres
```

2. Schema Creation in pg4admin:

- a. `customers_raw` table using SQL:

```
CREATE TABLE customers_raw (  
  customer_id INTEGER,  
  email TEXT,  
  phone TEXT,
```

```

country_code TEXT,
updated_at TIMESTAMP,
created_at TIMESTAMP
);

```

```

INSERT INTO customers_raw (customer_id, email, phone, country_code, updated_at, created_at) VALUES
(101, 'John@example.co.m', '111-222-3333', 'US', '2025-07-01 10:15:00', '2025-01-01 08:00:00'),
(101, 'john.d@example.c.om', '(111)222-3333', 'usa', '2025-07-03 14:25:00', '2025-01-01 08:00:00'),
(102, 'alice@example.com', NULL, 'UnitedStates', '2025-07-01 09:10:00', NULL),
(103, 'michael@abc.com', '9998887777', NULL, '2025-07-05 11:00:00', '2025-03-01 10:00:00'),
(104, 'bob@xyz.com', NULL, 'IND', '2025-07-05 15:00:00', '2025-03-10 09:30:00'),
(104, 'bob@xyz.com', NULL, 'India', '2025-07-06 18:00:00', '2025-03-10 09:30:00'),
(106, 'duplicate@email.com', '1234567890', 'SINGAPORE', '2025-07-01 08:00:00', '2025-04-01 11:45:00'),
(106, 'duplicate@email.com', '123-456-7890', 'SG', '2025-07-10 12:00:00', '2025-04-01 11:45:00'),
(108, NULL, NULL, NULL, NULL, NULL);

```

Sample Output:

	customer_id integer	email text	phone text	country_code text	updated_at timestamp without time zone	created_at timestamp without time zone
1	101	John@example.co.m	111-222-3333	US	2025-07-01 10:15:00	2025-01-01 08:00:00
2	101	john.d@example.c...	(111)222-33...	usa	2025-07-03 14:25:00	2025-01-01 08:00:00
3	102	alice@example.com	[null]	UnitedStates	2025-07-01 09:10:00	[null]
4	103	michael@abc.com	9998887777	[null]	2025-07-05 11:00:00	2025-03-01 10:00:00
5	104	bob@xyz.com	[null]	IND	2025-07-05 15:00:00	2025-03-10 09:30:00
6	104	bob@xyz.com	[null]	India	2025-07-06 18:00:00	2025-03-10 09:30:00
7	106	duplicate@email.c...	1234567890	SINGAPORE	2025-07-01 08:00:00	2025-04-01 11:45:00
8	106	duplicate@email.c...	123-456-7890	SG	2025-07-10 12:00:00	2025-04-01 11:45:00
9	108	[null]	[null]	[null]	[null]	[null]

b. `order_raw` table:

```
CREATE TABLE orders_raw (  
  order_id INTEGER,  
  customer_id INTEGER,  
  product_id TEXT,  
  amount NUMERIC(10, 2),  
  created_at TIMESTAMP,  
  currency TEXT  
);
```

```
INSERT INTO orders_raw (order_id, customer_id, product_id, amount,  
  created_at, currency) VALUES  
(5001, 101, 'P01', 120.00, '2025-07-10 09:00:00', 'USD'),  
(5002, 102, 'P02', 80.5, '2025-07-10 09:05:00', 'usd'),  
(5003, 103, NULL, 200.00, '2025-07-10 09:15:00', 'INR'),  
(5004, 105, 'P99', NULL, '2025-07-10 09:20:00', NULL),  
(5002, 102, 'P02', 80.50, '2025-07-10 09:05:00', 'USD'),  
(5005, 106, 'P03', -50, '2025-07-10 09:25:00', 'SGD'),  
(5006, 107, 'NULL', 300.00, '2025-07-11 10:00:00', 'usd'),  
(5007, 108, 'P04', 500.00, '2025-07-11 10:15:00', 'EUR');
```

c. `products_raw` table:

```
CREATE TABLE products_raw (  
  product_id TEXT PRIMARY KEY,  
  product_name TEXT,  
  category TEXT,  
  active_flag CHAR(1)  
);
```

```
INSERT INTO products_raw (product_id, product_name, category, active_flag) VALUES  
( 'P01', 'keyboard', 'hardware', 'Y'),
```

```
('P02', 'MOUSE', 'Hardware', 'Y'),  
('P03', 'Monitor', 'Hardware', 'N'),  
('P04', 'PremiumCable', 'Accessory', 'Y');
```

d. `country_dim`

```
CREATE TABLE country_dim (  
    country_name TEXT,  
    iso_code CHAR(2) UNIQUE  
);
```

```
INSERT INTO country_dim (country_name, iso_code) VALUES  
('UnitedStates', 'US'),  
('India', 'IN'),  
('Singapore', 'SG'),  
('Unknown', NULL);
```

2.3.2. Pipeline Creation :

- Source = PostgreSQL
- Destination = Snowflake
- Ingestion mode must be Logical Replication
- On local Postgres DB:
 - Command to enter the docker container to run the ngrok util: `docker exec -it mydatabase /bin/bash`
 - Using ngrok command to expose the DB to internet: `ngrok tcp 5432`

```
ngrok (Ctrl+C to quit)

Call internal services from your gateway: https://ngrok.com/r/http-request

Session Status      online
Account             Siddharth Kakade (Plan: Free)
Version             3.30.0
Region              India (in)
Latency             28ms
Web Interface        http://127.0.0.1:4040
Forwarding           tcp://0.tcp.in.ngrok.io:17339 -> localhost:5432

Connections          ttl    opn    rt1    rt5    p50    p90
                    0      0      0.00   0.00   0.00   0.00
```

- On Hevo UI:

Copy from Existing Sources ▾ Draft saved

Pipeline Name *
PostgreSQL Source 2

A unique name for your Pipeline

Database Host *
0.tcp.in.ngrok.io
10.123.1.001 or postgresql-replica.westeros.inc. [Learn more](#) →

Database Port *
17339 
The port on which the database is accepting connections. [Need help?](#) →

Database User *
admin

Database Password *
..... 

Select an Ingestion Mode SEE LESS ^

☒ **Logical Replication** Recommended
Data is ingested using PostgreSQL Write Ahead Logs (WALs). This is the most efficient way to set up real-time change data capture. Deletes are also replicated to the Destination.

☐ **Table**
Data is ingested by running SQL queries on your tables. This mode provides change data capture through modified/updated timestamp columns. Deletes are not replicated to the Destination.

☐ **Custom SQL**
Data is ingested by running a custom SQL query provided by you on the database. With this mode, you can replicate database views or selective data sets queried through the SQL query. Deletes are not replicated to the Destination.

- Connectivity issue: Although I am using the same container for the database and attempting to connect from a different pipeline, I keep encountering errors despite the container having all the necessary configurations and connectivity set up.

❗ Connecting to host

Unable to connect. Check the database host and port information. Ensure that the database host is accessible over the internet. If you are using a VPN or an on-premise database, whitelist Hevo's IP addresses. [Learn More >](#)

- I tried multiple times, but I was unable to get any output. I noticed that whenever I tried to create the pipeline, the latency of the ngrok tunnel increased significantly.

SQL Queries



Note - I was not able to reach the desired result of SQL script for cleaning the data but I've provided sample code that can be used

- **Deduplicate Customers**

- Keep only the most recent record for each customer_id

```
SELECT
  customer_id,
  email,
  phone,
  country_code,
  updated_at,
  created_at
FROM (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY updated_at DESC, created_at DESC
    ) as rn
  FROM
    customers_raw
) AS ranked_customers
```

```
WHERE
    rn = 1;
```

- Standardize emails to lowercase.

```
SELECT
    email AS original_email,
    LOWER(email) AS standardized_email
FROM
    customers_raw;
```

- Standardize phone numbers into a 10-digit format, or mark as "Unknown" if invalid or missing

```
SELECT
    customer_id,
    phone AS original_phone,
    CASE
        WHEN phone IS NOT NULL THEN
            REPLACE(
                REPLACE(
                    REPLACE(
                        REPLACE(phone, '-', ''),
                        '(', ''),
                        ')', ''),
                ' ', '')
        ELSE NULL
    END AS cleaned_phone_temp,
    CASE
        WHEN LENGTH(
            REPLACE(
                REPLACE(
                    REPLACE(
```

```

        REPLACE(phone, '-', ''),
        '(', ''),
        ')', ''),
        ' ', '')
    ) = 10 THEN
        REPLACE(
            REPLACE(
                REPLACE(
                    REPLACE(phone, '-', ''),
                    '(', ''),
                    ')', ''),
                ' ', '')
            ELSE 'Unknown'
        END AS standardized_phone
FROM
    customers_raw;

```

- **Fix <null>s & Country Issues**

- Standardize country_code values using country_dim.

```

WITH standardized_customer_raw AS (
    SELECT
        customer_id,
        email,
        phone,
        CASE
            WHEN LOWER(TRIM(country_code)) IN ('usa', 'unitedstates', 'u
nited states') THEN 'US'
            WHEN LOWER(TRIM(country_code)) IN ('ind', 'india') THEN 'IN'
            WHEN LOWER(TRIM(country_code)) IN ('singapore') THEN 'S
G'
            WHEN TRIM(UPPER(country_code)) IN ('US', 'IN', 'SG') THEN T
RIM(UPPER(country_code))
            ELSE country_code
        END AS standardized_code,

```



```

        updated_at,
        COALESCE(created_at, '1900-01-01 00:00:00') AS clean_created_
at
    FROM
        customers_raw
)
SELECT
    t1.customer_id,
    t1.email,
    t1.phone,
    COALESCE(t2.iso_code, 'NULL') AS final_iso_code,
    t1.updated_at,
    t1.clean_created_at
FROM
    standardized_customer_raw t1
LEFT JOIN
    country_dim t2

    ON t1.standardized_code = t2.iso_code OR t1.standardized_code = t
2.country_name;

```

- **Clean Orders**

- Remove exact duplicate rows.

```

SELECT DISTINCT
    order_id,
    customer_id,
    product_id,
    amount,
    created_at,
    currency
FROM
    orders_raw;

```

- Handle invalid amounts:

- If amount is negative, replace with 0.

```
SELECT
  order_id,
  customer_id,
  product_id,
  CASE
    WHEN amount < 0 THEN 0
    ELSE amount
  END AS amount,
  created_at,
  currency
FROM
  orders_raw;
```

- If amount is <null>, replace with a reasonable fallback (e.g., median of customer's transactions).

- Standardize currency codes to uppercase.

```
SELECT
  order_id,
  customer_id,
  product_id,
  amount,
  created_at,
  UPPER(currency) AS currency
FROM
  orders_raw;
```

- Create a derived column amount_usd by converting all amounts into USD using predefined conversion rates.

```

SELECT
  order_id,
  customer_id,
  product_id,
  amount,
  created_at,
  UPPER(currency) AS currency,

  (

    CASE
      WHEN amount < 0 THEN 0
      ELSE amount
    END
  *

    CASE
      WHEN UPPER(currency) = 'INR' THEN 0.012
      WHEN UPPER(currency) = 'SGD' THEN 0.74
      WHEN UPPER(currency) = 'EUR' THEN 1.07

      ELSE 1
    END
  ) AS amount_usd
FROM
  orders_raw;

```

- **Clean Products**

- Standardize product names (capitalize properly).
- Standardize category names (In "Title Case").
- If a product is inactive (active_flag = 'N'), mark it as "Discontinued Product"

```

SELECT
    product_id,
    INITCAP(LOWER(product_name)) AS product_name,
    INITCAP(LOWER(category)) AS category,
    CASE
        WHEN active_flag = 'N' THEN 'Discontinued Product'
        ELSE 'Active'
    END AS product_status
FROM
    products_raw;

```

- **JOIN the resultants**

- Produce a unified dataset joining customers, orders, and products.

- If customer_id does not exist in cleaned customers, mark email as "Orphan Customer".

- If product_id is missing or invalid, mark it as "Unknown Product".

- **Edge Cases**

- Customers with completely <null> records should be marked as "Invalid Customer".

- Orders referencing non-existent customers or products should still appear in the final dataset, with appropriate placeholders.

- Mixed currency handling should be consistent across all rows.

